implementation-plan

PROVENANT — MVP Implementation Plan

6-Hour Software Build (2 tracks, parallel) + 3-Hour Hardware Build (1 track)

This plan assumes a team of **5**: 2 people on Track A (Decision Validity Engine), 2 people on Track B (Research Sleeve + dashboard), 1 person on Hardware (the Vault). If your team is smaller, see the "Solo/duo fallback" note at the end of each hour block — tracks can be time-sliced instead of parallelized.

The hardware track (3h) runs **inside** the software timeline (it needs to be integrated by hour 5), it is not sequential-after.

---

0. GUIDING RULE FOR A 6-HOUR MVP

Everything is real except the data feed and the LLM calls, and even those are real-shaped. Use simulated/replayed market data (deterministic, seedable, scriptable for the demo) instead of a live exchange feed — live feeds add network-flakiness risk with zero demo upside. Use one or two real Claude API calls for the *explanation text* only (evidence weighting narrative, hypothesis description) — never for core numeric logic (Validity score, backtest stats), which must be deterministic and fast so the demo is reliable.

Cut list is decided now, not at hour 5. See Section 7.

---

1. TEAM & TRACK SPLIT

Track	Owns	People
A — Inner Loop	Data ingestion, Evidence layer, Opportunity generation, Risk/Allocation, Decision Validity Engine, Execution simulator	2
B — Outer Loop + Dashboard	Outcome monitoring, Failure analysis, Pattern detection, Research Sleeve, Backtest/OOS/promotion logic, React dashboard (renders both loops)	2
Hardware — The Vault	M5StickC Plus2 firmware, commit-reveal state machine, backend `/experiment` endpoints it talks to	1

Tracks A and B integrate through **one shared contract**: the `Decision` and `FailureEvent` JSON schemas (Section 4). Agree on these in the first 20 minutes and do not change field names after hour 1 — this is what lets two tracks build in parallel without blocking each other.

---

2. TECH STACK (deliberately minimal)

Layer	Choice	Why for 6 hours
Backend	Python 3.11 + FastAPI, single service, `asyncio` background tasks	One process, no distributed infra to debug; async fits the "continuous loop" model directly
Internal event bus	In-process `asyncio.Queue` per stage (ingestion→interpretation→...)	Kafka/Redis Streams add setup + failure surface with no demo-visible benefit at this scale
Database	SQLite via SQLAlchemy (single file)	Zero setup, inspectable with any SQLite browser during debugging, trivially resettable between demo runs
Frontend	React + Vite + Tailwind, native WebSocket client, Recharts	Fast scaffold, live-updating Validity gauges and pattern charts with minimal custom code
Realtime push	FastAPI WebSocket endpoint, one topic per track	Simplest way to stream Validity(t) and Vault state to the dashboard live
Data	Python generator script: seeded synthetic price/order-book/volatility ticks + a scriptable news-event injector	Reliable, replayable, demo-safe; no external API dependency during the pitch
LLM	Claude API (`claude-sonnet-4-6`), 1–2 calls per decision/hypothesis, **text explanation only**	Keeps novel logic deterministic and fast; LLM only narrates, never decides
Backtest/stats	pandas + numpy + scipy (t-test/Sharpe/p-value), pre-generated historical CSV (1–3 months, 2–3 assets)	No live data dependency for the Research Sleeve; walk-

Layer	Choice	Why for 6 hours
		forward split is just index slicing
Hardware firmware	Arduino C++ (M5StickC Plus2 SDK) or UIFlow MicroPython	C++ if the hardware owner is comfortable with it (more control over display/timer reliability); MicroPython if faster to iterate under time pressure — **decide in hour 0, don't switch later**
Hardware↔backend link	WiFi, HTTP polling every 1–2s (`GET /vault/state/{experiment_id}`)	Simpler and far more demo-reliable than sockets/MQTT on conference WiFi; backend is authoritative regardless

---

3. REPO STRUCTURE

```
provenant/
├── README.md                    # one-paragraph pitch + how to run
everything for judges
├── docker-compose.yml           # optional: spins up backend+frontend
together for demo laptop
├── .env.example
│
├── backend/
│   ├── pyproject.toml / requirements.txt
│   ├── app/
│   │   ├── main.py              # FastAPI app, WebSocket routes, startup
wiring
│   │   ├── config.py            # human-defined boundaries: max capital,
exposure, risk, cost limits
│   │   │
│   │   ├── ingestion/           # TRACK A
│   │   │   ├── simulator.py     # seeded synthetic price/orderbook/vol
generator
│   │   │   ├── news_injector.py  # scriptable news/event injection (for the
demo)
│   │   │   └── event_bus.py     # asyncio.Queue wiring between stages
│   │   │
│   │   ├── evidence/             # TRACK A
│   │   │   ├── models.py        # EvidenceNode, decay functions
```

```
│   │   │   ├── decay.py          # per-type decay curves
│   │   │   └── trust.py          # DataSourceTrust scoring
│   │   │
│   │   ├── opportunities/         # TRACK A
│   │   │   ├── generator.py      # reads Strategy Pool + evidence → Opportunity
│   │   │   └── templates.py      # structured hypothesis/strategy template definitions
│   │   │
│   │   ├── risk/                  # TRACK A
│   │   │   ├── evaluator.py      # downside, slippage estimate, exposure impact
│   │   │   └── allocator.py      # capital allocation under constraints
│   │   │
│   │   ├── decisions/             # TRACK A — CORE NOVEL MECHANISM (inner loop)
│   │   │   ├── models.py         # Decision, DecisionValidityHistory
│   │   │   ├── validity_engine.py # live Validity(t), adaptive threshold τ
│   │   │   └── actions.py        # HOLD/REDUCE/PAUSE/CANCEL/REVERSE dispatch
│   │   │
│   │   ├── execution/             # TRACK A
│   │   │   ├── simulator.py      # paper-trading fill engine, slippage model
│   │   │   └── validator.py      # pre-fill liquidity/cost re-check
│   │   │
│   │   ├── outcomes/              # TRACK B
│   │   │   ├── monitor.py        # tracks realized outcome vs. expected
│   │   │   └── failure_analysis.py # tags WHY a decision failed
│   │   │
│   │   ├── research_sleeve/       # TRACK B — CORE NOVEL MECHANISM (outer loop)
│   │   │   ├── pattern_detector.py # recurring failure clustering → ResearchTrigger
│   │   │   ├── hypothesis.py      # generates candidate from structured template space
│   │   │   ├── backtest.py        # historical backtest
│   │   │   ├── validation.py      # out-of-sample / walk-forward + stats
│   │   │   ├── promotion_gate.py  # statistical bar → promote/reject
│   │   │   └── strategy_pool.py   # live pool read by opportunities/generator.py
│   │   │
│   │   ├── vault/                 # HARDWARE INTEGRATION (backend side)
│   │   │   ├── commit.py          # canonical experiment → SHA-256 commit
│   │   │   ├── lock_state.py      # locked_at/unlock_at, reject_configuration_change()
│   │   │   └── router.py          # /vault/commit, /vault/state/{id}, /vault/reveal
│   │   │
│   │   ├── audit/                 # SHARED
```

```
│   │   │   └── ledger.py          # immutable decision/experiment log for
the dashboard
│   │   │
│   │   └── ws/                    # SHARED
│   │       └── broadcaster.py     # pushes live state to frontend over
WebSocket
│   │
│   └── tests/
│       ├── test_validity_engine.py
│       ├── test_promotion_gate.py
│       └── test_vault_commit.py
│
├── frontend/
│   ├── package.json
│   └── src/
│       ├── main.tsx
│       ├── ws/useLiveFeed.ts       # WebSocket hook, shared by both loop
views
│       ├── views/
│       │   ├── InnerLoopView.tsx   # Decision cards, evidence nodes,
Validity gauge, audit trail
│       │   ├── OuterLoopView.tsx   # Pattern detection, hypothesis cards,
Vault status, promotion
│       │   └── DashboardShell.tsx  # layout tying both views + human-
boundary config panel
│       └── components/
│           ├── ValidityGauge.tsx
│           ├── EvidenceNodeCard.tsx
│           ├── VaultTimer.tsx      # mirrors the M5Stick's state for anyone
not near the device
│           └── AuditTrail.tsx
│
├── hardware/
│   ├── README.md                  # wiring, flashing instructions
│   ├── firmware/
│   │   └── vault_device.ino       # (or .py if MicroPython)
COMMIT/LOCK/REVEAL state machine
│   └── docs/
│       └── display_states.md      # exact screen layouts per state (from
the idea doc)
│
├── data/
│   ├── historical/                # 2–3 assets, 1–3 months OHLCV for
backtest track
│   └── seeds/                     # deterministic scenario scripts for the
demo
│
└── docs/
    ├── idea.md
    ├── implementation-plan.md
```

```
    ├── api-contract.md            # frozen after hour 1 — see Section 4
    └── demo-script.md
```

---

4. THE SHARED CONTRACT (lock this in the first 20 minutes)

Track A and Track B must agree on these shapes before writing code, since B's Failure Analysis consumes A's Decision output, and A's Opportunity Generator consumes B's Strategy Pool output.

```
// Decision (emitted by Track A, consumed by Track B)
{
  "decision_id": "uuid",
  "opportunity_id": "uuid",
  "asset": "string",
  "action": "BUY | SELL",
  "evidence_nodes": [
    {"id": "uuid", "type": "news|orderbook|volatility|momentum", "weight":
0.35, "source": "string", "captured_at": "iso8601"}
  ],
  "validity_score": 0.91,
  "validity_threshold": 0.60,
  "status": "open|reduced|paused|cancelled|reversed|closed",
  "strategy_template_id": "string"
}

// FailureEvent (emitted by Track A/B outcome monitor, consumed by Research
Sleeve)
{
  "decision_id": "uuid",
  "strategy_template_id": "string",
  "regime": "high_vol|low_vol|trending|illiquid",
  "invalidation_cause":
"evidence_contradicted|evidence_decayed|trust_downgraded|execution_blocked",
  "dominant_evidence_type": "news|orderbook|volatility|momentum",
  "timestamp": "iso8601"
}

// StrategyPool entry (emitted by Research Sleeve, consumed by Opportunity
Generator)
{
  "strategy_template_id": "string",
  "params": {"confirmation_delay_sec": 300},
  "status": "active|retired",
  "promoted_at": "iso8601",
  "oos_sharpe": 1.42,
```

```
  "p_value": 0.018
}
```

Put this in `docs/api-contract.md` verbatim and treat it as frozen after hour 1.

---

5. HOUR-BY-HOUR SOFTWARE PLAN (6 HOURS)

Hour 0 (0:00–0:20) — Everyone

- Agree on the API contract above.
- Agree on: 2 assets, 3 evidence types for MVP (news, orderbook-imbalance, momentum — volatility used as a regime modifier, not a 4th weighted node, to save time), 1 strategy template family with 2 parameter variants (immediate entry vs. delayed entry).
- Scaffold repo per Section 3; everyone `git clone` + confirm backend/frontend boot.

Hour 1 (0:20–1:20)

Track A: Build `ingestion/simulator.py` (seeded price/orderbook ticks) + `news_injector.py` (script-triggerable events). Build `evidence/models.py` + `decay.py` with 3 hardcoded decay curves (news: 4 min half-life, orderbook: 20 sec, momentum: 90 sec).
Track B: Scaffold FastAPI WebSocket broadcaster (shared, but B owns since dashboard consumes it first). Load `data/historical/` CSVs, write a stub `backtest.py` that computes Sharpe/p-value on a slice — get this working end-to-end on dummy data immediately, don't wait for real decisions to exist.
Hardware: (see Section 6, runs independently) — hour 0–1 is firmware skeleton + display states.

Hour 2 (1:20–2:20)

Track A: `opportunities/generator.py` (rule-based: if news+momentum align → propose opportunity, cite evidence with fixed weights) + `risk/evaluator.py` + `risk/allocator.py` (simple: equal-weight within max-exposure cap, reject if downside estimate breaches risk limit).
Track B: `research_sleeve/pattern_detector.py` — a simple counter: 3+ `FailureEvent`s with the same `strategy_template_id` + `regime` within the session window fires a `ResearchTrigger`. Also start `promotion_gate.py` (thresholds: p < 0.05, OOS Sharpe > 0.8, decay < 15%).
Checkpoint (2:20): Track A produces a real `Decision` JSON on the bus; Track B's WebSocket broadcaster picks it up and logs it. **This is the first integration test — do not skip it.**

Hour 3 (2:20–3:20)

Track A: `decisions/validity_engine.py` — THE CORE MECHANISM. Recompute `validity_score` on every tick from evidence decay + a hardcoded "contradiction" rule (a news_injector event tagged `contradicts: decision_id` zeroes that node's weight instantly). Adaptive threshold: two fixed values (0.60 in `high_vol` regime, 0.45 otherwise) is enough — don't build a continuous function under time pressure. `decisions/actions.py` dispatches REVERSE/CANCEL/etc. and writes to `audit/ledger.py`.
Track B: `outcomes/failure_analysis.py` (consumes the audit ledger, emits `FailureEvent` per the contract) + `research_sleeve/hypothesis.py` (from the ResearchTrigger, deterministically construct the "delayed entry" variant — this does NOT need an LLM, it's a template parameter swap).
Checkpoint (3:20): Inject one scripted contradicting news event manually → confirm Validity visibly drops and a REVERSE action is logged end-to-end.

Hour 4 (3:20–4:20)

Track A: `execution/simulator.py` + `execution/validator.py` (pre-fill liquidity/slippage check — can be a simple function of orderbook depth). Wire the Vault commit call: when Research Sleeve wants to test a hypothesis, Track A's execution module is untouched (firewall) — this is Track B's job to call `vault/commit.py`.
Track B: `research_sleeve/backtest.py` + `validation.py` real implementation on historical CSVs (walk-forward split: first 70% train window conceptually irrelevant here since it's a rule-parameter test, not ML — just run both parameter variants on out-of-sample slices and compare Sharpe/p-value). Wire `vault/commit.py` → POST to hardware's backend endpoint, and `vault/lock_state.py` enforcing `reject_configuration_change()` for 60s server-side (independent of whether the physical device is even connected — **critical for demo safety**, see Section 7).
Checkpoint (4:20): Full outer loop dry run without hardware: trigger → hypothesis → (simulated 60s lock) → backtest → reveal → promotion decision, logged.

Hour 5 (4:20–5:20)

Everyone + Hardware owner joins backend integration:

- Wire the real M5StickC Plus2 into `vault/router.py` (device polls `/vault/state/{id}`).
- Frontend: `InnerLoopView.tsx` (Validity gauge, evidence cards, audit trail) and `OuterLoopView.tsx` (pattern panel, hypothesis card, Vault mirror timer, promotion result) — split these two views between the two Track B people if the hardware owner is now free to help wire WebSockets.
- Full run-through #1: seeded scenario script from `data/seeds/` end-to-end, both loops, physical device included.

Hour 6 (5:20–6:00)

- Bug fixes only. No new features after 5:40.

- Full run-through #2 and #3 (rehearse the demo script exactly as it will be presented, timed).
- Reset scripts (`data/seeds/reset.py`) tested — you must be able to reset DB + Vault state between the two run-throughs and the live judge demo without restarting services.

Solo/duo fallback: if fewer than 5 people, cut in this order without breaking the story: skip volatility as a separate regime input (hardcode 2 regimes flagged manually in the seed script) → skip real historical backtest, use 1 pre-computed comparison table for the two template variants → skip Track A's execution/validator.py depth check, use a fixed slippage constant. **Never cut the Validity Engine or the Vault** — they are the entire idea.

---

6. HOUR-BY-HOUR HARDWARE PLAN (3 HOURS, overlapping hours 0–3 of the software timeline)

Hour 1 (0:00–1:00) — Firmware skeleton

- Flash a "hello world" to confirm M5StickC Plus2 + IDE (Arduino or UIFlow) toolchain works — do this literally first, before writing any state-machine code; USB driver issues are the #1 hardware-track time sink.
- Build the 3 static display screens exactly as specified in the idea doc: 🟡 COMMIT ("NEW EXPERIMENT / EXP #0187 / COMMITTING..."), 🔴 LOCK ("EXPERIMENT LOCKED / 00:42 / DO NOT MODIFY / Hypothesis ✓ Params ✓ Dataset ✓"), 🟢 REVEAL ("LOCK EXPIRED / REVEALING... / COMMIT ✓ RESULT ✓ / VERIFIED"). Hardcode dummy data first — get the visuals right before wiring networking.

Hour 2 (1:00–2:00) — Networking + polling

- Connect to hackathon WiFi (test this early — venue WiFi is the #2 time sink; have a phone hotspot as backup, confirmed working, by end of this hour).
- Implement HTTP GET polling of `/vault/state/{experiment_id}` every 1–2s.
- State machine driven entirely by server response fields (`state: committing|locked|revealing|verified` , `seconds_remaining` , `commit_hash_short` , `oos_sharpe` , `p_value`) — **the device has zero local logic about whether a lock is valid; it only renders what the backend tells it.** This matches the "backend is authoritative" design from the idea doc and means the device can be reflashed or restarted anytime without corrupting state.

Hour 3 (2:00–3:00) — Integration + resilience

- Wire against Track A/B's real `/vault` endpoints (backend team should have a stub responding by hour 2 of the software timeline — coordinate this explicitly, it's the one hard cross-track dependency).

- Add a visible truncated commit hash on the LOCK screen (first 6 + last 4 hex chars is plenty — full 64-char hash won't fit and doesn't add demo value).
- **Failure mode handling:** if WiFi drops mid-demo, the device should show a clear "RECONNECTING" state rather than freezing on stale data — this is a 15-minute addition that saves the entire demo if venue WiFi hiccups.
- Battery-check and have a USB power bank taped to the device for the live demo — do not run the demo on internal battery alone.

By end of hour 3 (hardware) / hour 5 (software): the device must be able to sit next to the laptop, be handed a real commit from the backend, count down, and flip to REVEAL/VERIFIED without anyone touching it. That handoff is rehearsed in software Hour 5–6.

---

7. CRITICAL DEMO-SAFETY DECISION: BACKEND IS AUTHORITATIVE, ALWAYS

Per the idea doc, the lock **must** be enforced server-side (`reject_configuration_change()`) independent of the physical device. Build and test this in software Hour 4 **before** the hardware is even wired in. This means: if the M5StickC Plus2 fails at demo time (dead battery, WiFi drop, dropped on the floor), the software story still works end-to-end using `VaultTimer.tsx` on the dashboard as a fallback visual — the judges lose the "wow, a physical device" moment but not the underlying architecture demo. **Never make the live demo hard-dependent on hardware working.**

---

8. CUT LIST (decide now, don't debate mid-build)

Cut without hesitation if behind schedule, in this order:

1. Real LLM-generated explanation text → replace with templated strings (`f"Decision invalidated: {evidence_type} evidence {cause}"`). Costs nothing story-wise.
2. Continuous adaptive threshold function → two hardcoded regime thresholds.
3. 3rd asset → 2 assets only.
4. Real historical CSV backtest → 1 pre-computed comparison result, still routed through the real Vault commit-reveal flow (the *flow* is the demo, not the statistical engine).
5. Slippage as a function of live orderbook depth → fixed constant per asset.

Never cut:

- The live Validity(t) recomputation and autonomous action dispatch (Inner Loop) — this is the entire Level-1 novelty.

- The commit→lock→reveal→verify flow, even in software-only fallback — this is the entire anti-p-hacking novelty and the hardware story.
- The Recurring Pattern Detection → Research Trigger connection — this is what makes the two loops *one system* instead of two demos glued together.

---

9. WORKFLOW / GIT PROCESS

- **Branching:** `main` is always demo-runnable. Each track works on `track-a` and `track-b` (hardware on `hardware`), merging to `main` at the 4 checkpoints listed in Section 5 (end of hour 2, 3, 4, 5) — not continuously. Short-lived feature branches off the track branch are fine but merge back within the hour.
- **No rebasing/force-pushing after hour 3** — resolve conflicts by talking, not git archaeology; you don't have time for it.
- **Contract changes:** any change to the JSON shapes in Section 4 after hour 1 requires a 2-minute sync with both tracks before merging — this is the one rule everyone must follow, since a silent field rename breaks the other track invisibly.
- **Demo data resets:** `data/seeds/reset.py` truncates SQLite tables and re-seeds the deterministic scenario; run this before every rehearsal and before the live judge run so timings are reproducible.
- **Rehearsal cadence:** hour 5:20 (first full run), hour 5:45 (second, timed with a stopwatch against the 12-phase demo script in `docs/demo-script.md`), hour 5:55 (final check: hardware battery, WiFi fallback, laptop screen-share settings).

---

10. DEMO SCRIPT TIMING (target: under 2m30s live, matches idea.md Section 9)

Phase	Target time	What's on screen
1–2	0:00–0:20	Decision created, Validity 0.91, evidence cards visible
3–4	0:20–0:45	Contradicting event injected → Validity collapses → autonomous REVERSE + audit explanation
5–6	0:45–1:00	Fast-forward to pattern detected (pre-seeded prior failures)
7	1:00–1:15	Hypothesis card appears

Phase	Target time	What's on screen
8–9	1:15–2:15	**Physical device commits and counts down (real 60s, or narrated/sped in rehearsal if judges are time-constrained — confirm which with organizers beforehand)**
10–11	2:15–2:30	Vault reveals VERIFIED, dashboard shows PROMOTED
12	2:30–2:45	New event injected, dashboard shows the new strategy template being used

If the 60-second physical lock is too long for the judging slot, say so explicitly in the pitch ("in production this window is calibrated to the evidence class; for the demo we've set it to 60 seconds so you can watch it") rather than secretly shortening it — the visible wait *is* the point of the mechanism.